Tutorial 3: Memory Management

Chan Jing Yi | SUOL2500321 | BCL1213 Operating Systems

1. Define memory management and state two objectives.

Memory management is the operating system function that tracks, allocates, protects and releases main memory for processes and the operating system.

Two objectives are to use available memory efficiently and to protect each process from unauthorised access by other processes.

2. List four levels of the memory hierarchy from fastest to slowest.

CPU registers, cache memory, main memory (RAM), and secondary storage. Speed varies within each level, but this is the usual order.

3. Why are registers and cache closer to the processor than secondary storage?

The CPU uses registers and cache to obtain frequently needed instructions and data quickly. Their proximity and circuit design reduce access latency, helping the CPU avoid waiting. Secondary storage provides much larger, persistent capacity at a lower cost per byte, but access is slower.

4. Differentiate the memory allocation techniques.

Fixed partitioning divides memory into partitions of predetermined sizes. A process is placed in a suitable partition, and unused space within that partition is internal fragmentation.

Dynamic partitioning creates variable sized partitions according to process needs. This can reduce internal waste, but free gaps between allocations can cause external fragmentation.

Noncontiguous allocation places parts of a process in separate locations rather than one continuous block. Paging and segmentation are examples. Paging uses equal sized pages and frames, whereas segmentation uses variable sized logical segments.

5. Differentiate internal and external fragmentation and give one cause of each.

Internal fragmentation is unused space inside an allocated block. It can occur when a process is smaller than its fixed partition or when its last page does not fill a frame.

External fragmentation is free memory scattered between allocated blocks. It can occur when variable sized allocations are created and released over time, leaving gaps too small or dispersed for a new contiguous allocation.

6. Fixed partitioning calculations

P1: 100 KB allocated to a 90 KB process. Unused memory = 100 − 90 = 10 KB.

P2: 200 KB allocated to a 150 KB process. Unused memory = 200 − 150 = 50 KB.

P3: 300 KB allocated to a 250 KB process. Unused memory = 300 − 250 = 50 KB.

P4: 400 KB allocated to a 350 KB process. Unused memory = 400 − 350 = 50 KB.

Total internal fragmentation = 10 + 50 + 50 + 50 = 160 KB.

7. Pages and frames

a. Number of pages = logical address space ÷ page size = 32 KB ÷ 4 KB = 8 pages.

b. Number of frames = physical memory ÷ frame size = 16 KB ÷ 4 KB = 4 frames.

c. Pages and frames have the same size so that any page can fit into any frame. The page offset remains unchanged during address translation, while the page number is mapped to a frame number.

8. Segmentation address calculations

Treat each limit as the length of the segment. An offset is valid when 0 ≤ offset < limit. For a valid address, physical address = base address + offset.

a. Segment 0, offset 200: 200 < 500, so the address is valid. Physical address = 1200 + 200 = 1400.

b. Segment 1, offset 750: 750 < 800, so the address is valid. Physical address = 2500 + 750 = 3250.

c. Segment 2, offset 350: 350 is greater than the limit of 300, so the address is invalid. Valid offsets are 0 through 299.

d. Segment 3, offset 500: 500 < 600, so the address is valid. Physical address = 5200 + 500 = 5700.

9. Compare paging and segmentation.

Paging divides logical memory into fixed size pages and physical memory into frames of the same size. Segmentation divides a program into logical units, such as code, data or a stack, whose sizes can vary.

Paging avoids external fragmentation in physical memory because any page can occupy any available frame. It can still cause internal fragmentation, especially in the final page. Segmentation with contiguous placement of each segment can cause external fragmentation because differently sized segments leave gaps.

10. Moving an inactive process to secondary storage

a. The technique is swapping.

b. The operating system moves an inactive process out of RAM to a designated area of secondary storage, freeing memory for other processes. When the process is needed again, the system brings it back into RAM before it resumes. Disk transfers are much slower than RAM access, so excessive swapping can reduce performance.

c. The medium term scheduler is responsible for suspending and later reintroducing processes in this model.

11. Identify the memory access methods.

a. Magnetic tape: sequential access. Data is read in order until the required location is reached.

b. RAM: random access. A location can be selected directly by its address without reading preceding locations.

c. Cache searched by contents: associative access. A tag or key is compared with stored entries to find a match.

d. Hard disk: direct access. The device moves to an approximate location and then locates the required data within the selected area.